神经网络设计与实践(I)
扩展性调研报告
图卷积神经网络

课程名： 神经网络设计与实践(I)　　**学　院：** 计算机学院
主　题： 图卷积神经网络　　**专　业：** 人工智能
姓　名： 徐陈天野　　**年　级：** 2019 级
学　号： 2019141460119　　**班　级：** 193040701

图卷积神经网络与
基于图卷积神经网络的引文节点数据分类研究

摘　要： 真实世界中，许多重要的数据都以图的形式存在，比如文献引用关系、蛋白质相互作用网、社交网络等等。相对于文本是一个序列、图片是一个矩阵来说，图这种数据结构更为多样复杂也更难处理。为了将神经网络运用在这类特殊的数据上，过去几年间一些论文提出了将神经网络一般化，以普适所有图数据的方法。而其中效果较好的一种即是图卷积神经网络。本调研报告尝试从数学原理出发，类比卷积神经网络，构建图卷积神经网络模型，并在 Cora 数据集上应用此网络，同时训练一个传统全连接神经网络进行对比实验，结果证明图卷积神经网络对 Cora 这类图数据的学习能力非常优秀，对图数据具有良好的适配效果，具有广阔的应用场景。

关键词： 图卷积神经网络；Cora 数据集；节点分类；图数据

目　录

摘要与关键词……01
一、引言……03
二、国内外研究现状……05
三、模型建立：从卷积到图卷积……06
四、编程实战：构建图卷积神经网络解决 Cora 数据集论文分类问题……19
五、总结与讨论……26
参考文献……29
附录：实验代码……31

一、引言

在深度学习技术兴起的前几年，图像、语音、文本等形式的数据都能在深度学习中被很好地应用，并获得了十分良好的效果。这促使大量相关应用进入了实用阶段，如人脸识别、语音助手、机器翻译等。这些领域的数据具有平移不变性。平移不变性使得我们可以在输入数据空间定义全局共享的卷积核，从而定义卷积神经网络。以图像数据为例，一张图片可以表示为欧氏空间中一组规则散布的像素点，平移不变性则表示以任意像素点为中心，可以获取相同尺寸的局部结构。基于此，卷积神经网络通过学习在每个像素点共享的卷积核来建模局部连接，进而为图片学习到意义丰富的隐层表示。

尽管传统的卷积神经网络在文本和图像领域带来提升，但是它仅能处理欧氏空间数据。深度学习一直无法很好地对另一类形式的数据——图数据进行有效的适配。实际上，真实世界中，许多重要的数据都以图的形式存在，比如文献引用关系、蛋白质相互作用网等等。同时作为一类主要用来描述关系的通用数据表示方法，图数据在产业界有着更加广阔的应用场景。

相对于文本是一个序列、图片是一个矩阵来说，图这种数据结构相对来说更为复杂。不同于图像和文本数据，图数据中每个节点的局部结构各异，这使得平移不变性不再满足。平移不变性的缺失给在图数据上定义卷积神经网络提出了挑战。上一次机器学习领域对图数据的关注还是在十几年前社交网络比较流行的时候。但是在过去一些年里，由于图数据的处理较为困难，很长一段时间内研究人员的关注点还是集中在别的方向。不过，可喜的是，近三年中，我们可以看到将神经网络应用在图数据上面这个研究方向在逐渐兴起。将深度学习技术的成功经验迁移到图数据的学习中来，是一种十分自然且必要的需求。在这样的背景下，研究人员开始关注如何在图上构造深度学习模型。借助于卷积神经网络对局部结构的建模能力及图上普遍存在的节点依赖关系，图卷积神经网络成为其中最活跃最重要的一支。图卷积神经网络的出现很好地填补了上述技术空白，实现了图数据与深度学习技术的有效结合，使得深度学习能够在图数据的相关应用场景中继续攻城略地。事实上，从 2018 年末至今的时间里，图卷积神经网络技术的相关应用和研究已经拓展到了极其广泛的领域，从视觉推理到开放的阅读理解问题，从药物分子研发到 5G 芯片设计，从交通流量预测到 3D 点云数据的学习，该项技术都展示出了极其重要且极具渗透性的应用能力，这种能力必将给产业界带来极高的应用价值。

图 1 典型的图数据——社交网络

图卷积神经网络（GCN）是一种基于深度学习的处理图域信息的方法。由于其较好的性能和可解释性，GCN 最近已经成为一种广泛应用的图分析方法。GCN 是受到卷积神经网络 CNN 的启发而出现的。卷积神经网络的强大表现力和高准确度使得其在许多研究领域取得了巨大成果，如图像识别、图像分割、语音识别等等。然而卷积神经网络只能在规则的欧几里得数据上运行，如图像等等。而图数据是非欧几里得数据，传统的卷积神经网络难以处理，于是借鉴卷积神经网络的思想，构建一个新的、适用于图数据的神经网络模型成了一个重要突破点。图卷积神经网络就在这种需求中应运而生。

近年来，随着人工智能的发展，人工智能子领域方面的论文层出不穷，例如神经网络、强化学习、规则学习等等。一般论文的搜集者在面对海量的论文时，常常只需要与自己研究相关的子主题的论文，对其他子领域的论文并没有较大兴趣，因此常常依赖于论文的预先主题分类来选择是否阅读，以节省时间、提升效率。幸运的是，在一些正规论文网站上，大部分人工智能论文被准确划分进对应的子主题，而论文类别是由作者填写的。可是即便如此，有些论文数据库因为种种原因，比如论文发表时相关主题并未出现或广为人知、标注子主题的人员并非作者而是不太专业的编辑人员等等，也有一些论文被粗糙地划分在人工智能大主题下；更有一些分类粗糙的论文数据库，并没有为人工智能子领域单独设主题，而仅有人工智能这一大主题，这时候就需要论文搜集者逐篇进行阅读来确认所属的子领域，不仅浪费宝贵的科研时间，更让文献搜集变得工程量巨大。

为了方便论文数据库对人工智能论文的（子）主题进行标注，并为文献检索者提供检索便利，我希望构建一个人工智能相关论文子主题分类模型。在基于传统的统计学习的分类算法进行文本主题分类的研究以及传统神经网络进行文本主题分类的研究基础上，我更进一步地考虑到论文含有引用关系这一特殊性特征，这些引用关系在同一大领域下的各个子主题之间和之内尤为频繁。引用关系或许对论文的主题分类也能提供一些有效信息，因为按照常理来讲，一篇论文引用的论文有极大可能与其主题相同或相近。基于这样的假设，本调研报告尝试在传统文本分类只进行孤立文本特征学习的基础上，引入论文的引用关系作为补充特征，与文本的特征一同进行学习。若把这些引用关系看作边，论文看作节点，就构成了一张图论中定义的图。

利用图卷积神经网络对论文及其引用关系构成的图数据进行学习，不仅考虑了文本各自孤立的特征，还创造性地考虑了论文数据有别于其他文本内容的特殊特征——引用关系，有望突破传统论文主题分类方法准确度不高的瓶颈。

综合以上两方面背景，本调研报告将引入图卷积层和图卷积神经网络这一全新的神经网络层和神经网络，并将其应用在论文特征与引用关系数据集 Cora 数据集上，完成论文的主题分类任务，并搭建训练一个传统全连接神经网络完成同样的分类任务作为对照实验，以对比观察图卷积机制对于图数据的学习效果。

由于图数据对于我们来说可能较为陌生，本调研报告首先探讨了什么数据可以表示成一张图；其次我们来对比一下图数据和其他的数据有何不同，以及为什么对于图数据，传统的神经网络不再适用；然后我们将会从数学原理出发，类比卷积神经网络，自然地引入图卷积神经网络；最后我们实现一个具体的图卷积神经网络，完成 Cora 数据集上的论文节点主题分类任务，并设置传统全连接神经网络作为效果对照，对比观察图卷积神经网络相较于传统神经网络在图数据适配上的优势。

二、国内外研究现状

图数据建模的历史由来已久。起初，研究人员关注统计分析的方法，这个时期没有机器学习模型的参与，如网页排序的常用算法 PageRank[1]、HITS[2]等。此外，研究人员也借用图谱理论的知识，如用拉普拉斯矩阵的特征值和特征向量做社区分析或者人群聚类[3]等。随着深度学习的崛起，研究人员开始考虑把深度学习的模型引入到图数据中，代表性的研究工作是网络嵌入（Network Embedding）[4]，即通过约束节点的邻近性为每个节点学习固定长度的表达，如 DeepWalk[5]、LINE[6]、node2vec[7]等。这 一时期，在解决具体的应用问题时，研究人员通常将其建模为两阶段问题，以节点分类为例，第一阶段为每个节点学习统一长度的表达，第二阶段将节点表达作为输入，训练分类模型。近年来，研究人员对图数据建模的关注逐渐转移到如何将深度学习的模型迁移到图数据上，进行端到端的建模，而图卷积神经网络则是其中最活跃的一支。

在建模图卷积神经网络时，研究人员关注如何在图上构建卷积算子。Bruna 等人[8]在 2013 年提出第一个图卷积神经网络，他们基于图谱理论从卷积定理出发，在谱空间定义图卷积。这一支后来发展为图卷积领域的谱方法。最初的谱方法具有时空复杂度较高的弊端，ChebNet[9]和 GCN[10]对谱方法中的卷积核进行参数化，大大降低了时空复杂度。这两个方法虽然被归为谱方法，但已经开始从空间角度定义节点的权重矩阵。在这两个方法的启发下，空间方法应用而生，开始考虑在节点域用注意力机制、序列化模型等建模节点间的权重。这一时期的图卷积神经网络在构建卷积算子过程中没有过多的考虑图特性，随着卷积算子的逐渐完善，人们开始考虑多样的图特性，如开始关注如何建模图上的高阶信息，并分别针对边上带特征的图、异质图等进行精细设计。 此外，如何训练更高效的图卷积神经网络也受到广泛关注。研究人员开始试图训练更深层的图卷积神经网络，以增强模型的泛化能力。同时，模型到大规模图的可扩展性以及训练的速度也是图卷积神经网络中非常重点的研究方向。

科技论文的分类任务在当前科技成果爆发式产出的时代尤为重要。现有有很多实际投入使用的工具与算法可以帮助文献搜集者更有效地搜寻目标主题文献。例如基于共引分析的 Histcite 和 CiteSpace 等工具[11]。Histcite 主要功能为统计分析，在引文网络的分析构建上架构较为清晰，可以清楚地发现引文网络中关键文献及其大概发展脉络；CiteSpace 功能类似，但更有一个最为突出的功能，即文献共被引分析，可以通过简单的方式找到引用次数较高的文献，同时通过共被引聚类分析，可以分析研究主要集中的主题，并找到每个聚类中的关键文献。

实际上，对论文的主题分类的实质就是对非结构化的文本进行分类，其相关算法一直被很多学者关注、研究和推进。1960 年到 2000 年之间，文本分类大多是基于浅层学习的，亦即基于统计的模型，如朴素贝叶斯法、决策树等等。而在文本的表示与特征选择上，则大多是基于词频统计的方法。杨清等人[12]使用 TF-IDF（term frequency – inverse document frequency）分数评估特征在一段文本中的重要性，并给与一定的重要性得分，把这段文本表示为关键特征词与 TF-IDF 值组成的元组的集合，最后使用感知机学习算法来确定文本的主题类别。S. Zhou 等人[13]采用 n-gram 来表示文本特征进行分层分类，也取得了较好的结果。二十世纪九十年代后期以来，文本分类方法中出现了大量机器学习的身影，而同期国内对其研究还有所欠缺。

主题模型起源于 S. C. Deerwester 等人[14]在 1900 年提出的隐含语义分析，在隐含语义分析的基础上，T. Hofmann[15]于 1999 年提出了第一个主题模型概率隐含语义分析。D. Blei 等人[16]于 2003 年提出隐含迪利克雷分布，弥补了主题模型概率隐含语义分析仍是不完整的概率模型的问题。隐含迪利克雷是目前最常用的主题概率模型，当今许多主题模型均由此发展而来，例如 ATM（Author Topic Model）、hLDA（Hierarchical Topic Model）、PAM（Pachinko Allocation Model）等等。

相较于传统的机器学习算法而言，尽管也存在一定的局限性，以深度学习为基础的分类方法在特征提取方面有相当大的优势。近年来研究和实践筛选出了许多代表性的深度学习算法如 RNN、CNN 等等。其中，以 LSTM 及其变形为代表的 RNN 不仅可以有效地捕获文本的长距离特征，还可以有效处理不定长的线性文本数据。但因为 RNN 要求计算当前时刻输出时要考虑上一时刻的隐藏状态，从而导致 RNN 难以实现并行计算，进而限制了运算速度。而 CNN 利用堆积的卷积层、非线性层、池化层等来自动提取特征，但是 CNN 在长距离特征提取上性能低于 RNN[17]。Tai 等人[18]改进了传统 LSTM 并提出 Tree-LSTM 模型，使其学习到更加丰富的语义表示。

披读文献，发现近年来国内外论文主题分类相关研究中，大部分都是基于孤立文本特征的分类模型训练，相关研究仅仅是局限于如何更好地生成文本的特征表示，亦或是单纯地基于论文的引用关系来进行聚类，鲜有同时考虑文本自身的特征以及论文引用关系的分类方法。究其原因，我认为，由于缺乏相关适配图数据的模型，导致过去对特征与关系进行同时学习较为困难，而今新兴的图神经网络恰恰填补了这一空白。将图神经网络运用于论文主题分类中，考虑孤立的论文自身特征，同时考虑论文的引用关系，极大可能获得比先前两种考量单独实践的效果更好的效果。本调研报告即基于此想法进行相关研究。

三、模型建立：从卷积到图卷积

本节我们将会从图数据出发，分析对于图数据定义一种新的模型的必要性，并且从数学原理角度进行分析，将图信号处理的基本理论，结合与卷积神经网络的适当类比，顺其自然地推广到图卷积神经网络中，最后构建出图卷积神经网络模型。

（一）图数据的构建

我们可能已经熟悉了某些类型的图数据，比如社交网络等等。然而，图是一种极其强大和普遍的数据表示方法，这里将展示两种我们可能认为不可以用图来建模的数据类型：图像和文本，是怎样转化为图数据的。虽然是反直觉的，但通过将图像和文本看作是图，可以更多地了解它们的对称性和结构，并建立对图数据的初步印象，这将有助于理解其他不太像网格的图数据，我们将在后面讨论。

（1）图视角看图像

我们通常认为图像是具有通道的矩形网格，将它们表示为数组（例如 244×244×3 的浮点数组）。另一种思考图像的方式是具有规则结构的图，其中每个像素代表一个节点，并通过边连接到相邻的像素。每个非边界像素正好有 8 个邻居，每个节点存储的信息是一个代表像素 RGB 值的 3 维矢量。

邻接矩阵是用来可视化图连通性的一种方法。我们首先对节点进行排序。在下图 2 的情况下，一个简单的 5×5 的笑脸图像中，共计有 25 个像素，每个像素视为一个节点，每个节点和它对应像素的上、下、左、右、左上、左下、右上、右下 8 个方向相邻的像素对应的节点相连，得到原始图像数据对应的图数据，如图 3 所示。如果两个节点被一条边相连，对应的 5×5 邻接矩阵的相应位置就为 1，否则为 0，如图 4 所示。

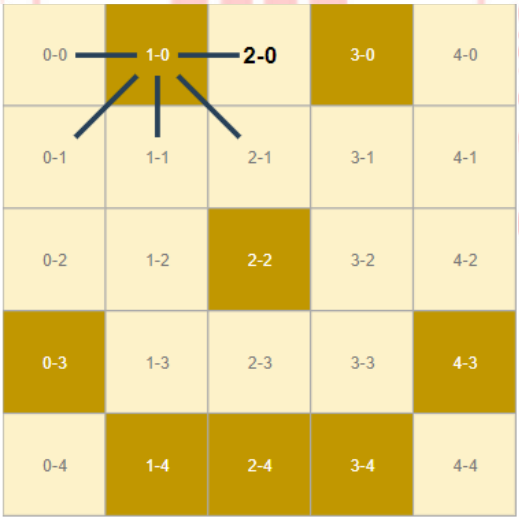

图 2 原始图像数据

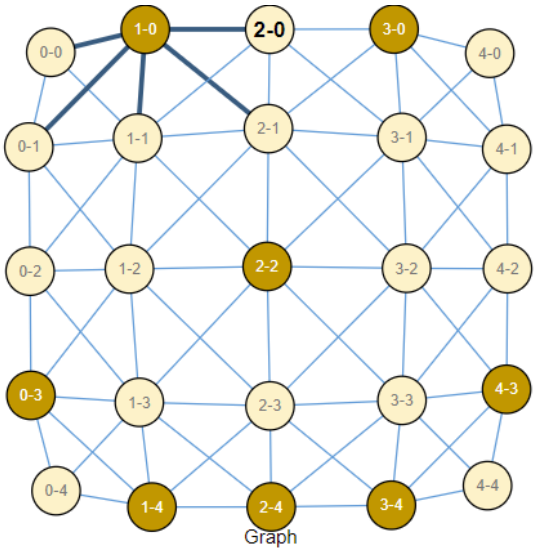

图 3 原始图像数据对应的图数据

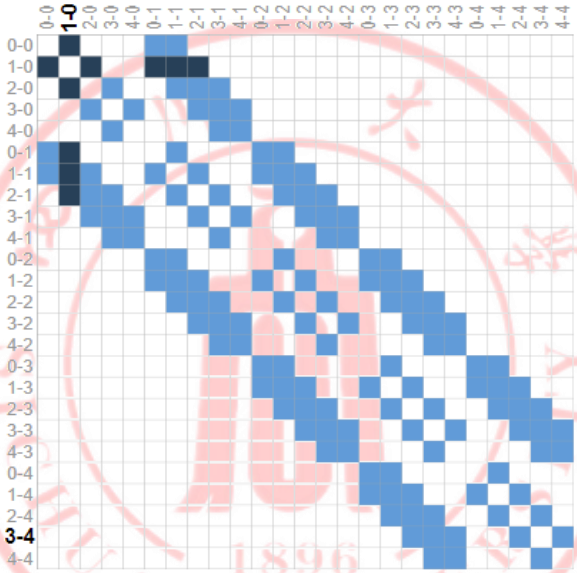

图 4 图数据对应的邻接矩阵

（2）图视角看文本

要将文本转化为图数据，我们可以将一段文本中每个字符、词或标记视为一个节点，并按前后顺序关系建立有向边。这就形成了一个简单的有向图，其中每个字符都是一个节点，并通过一条边与后面的节点相连。

例如，在图 5 的例子中，我们可以将一句话“Greetings from XU”拆成三个节点，每个节点分别代表一个词，并且按照前后出现顺序建立有向边，如图 5 所示。这样就构成了一个图，其对应的邻接矩阵如图 6 所示。

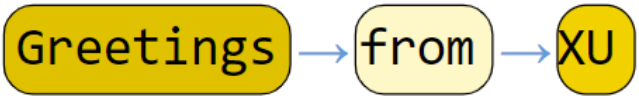

图 5 文本数据对应的图数据

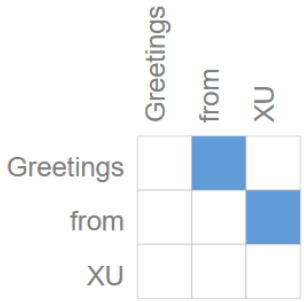

图 6 图数据对应的邻接矩阵

（3）其他常见的图数据与图数据的适配问题

通常情况下，在实际处理问题中，文本与图像并不会以这种图的方式进行表示，这会造成许多冗余信息，因为图像和文本的结构非常规则。我们可以看到，在图 4 和图 6 中，图像的邻接矩阵都是相同的带状结构，因为所有的节点都是以网格方式连接的；而文本数据的邻接矩阵只是一条对角线，因为每个词只与前一个词和后一个词相连接。由于这两种数据的高度规则性，我们不需要显式地定义出边，边的信息在这两种数据中是隐式给出的。

但是，在现实世界中，除了图像和文本外，还存在这许多并不是很规则的图结构，例如化学分子、社交网络、引文网络等等。这些数据与图像和文本相比，具有高度的不规则性，我们必须完整地表示出节点与边，否则原始数据的意义将被破坏。

然而，正因为这种不规则性，在图像和文本这些规则数据上适用的卷积神经网络等传统神经网络模型对于图数据就不再适用，在图像数据上卷积操作可以很简单地聚合邻居像素的信息，然而在图数据上，如果依旧使用这种方法显然不行。因为对于每一个节点来说，其邻居数量都是不同的，并不像图像一样，每个像素的邻居像素数量不变，因此对于图数据来说，固定大小的卷积核这种想法是行不通的。

虽然固定卷积核大小的想法在图数据上难以实现，我们依旧可以借鉴图像上卷积操作的思想，亦即聚合邻居节点的信息。我们基于这样的假设，认为邻居节点的信息对于自身节点的信息可以起到比较好的补充作用，这样就比孤立地考虑单个节点的信息对该节点的认识更加全面。就好像我们评价一个人，除了审视这个人自身的品质，还可以通过观察他的朋友的品质来作为对他自身认识的补充。

因此，我们需要设计一种类似于图像卷积的机制，能够将图数据中的邻居节点信息聚合到自身节点上，以充分利用自身节点特征之外的邻居特征作为自身特征的补充。

下面一节我们将基于以上想法，从我们的目标出发，进行适当的推导与尝试，构建出符合我们预期的“图卷积”模型。

（二）图卷积模型的构建

由于图卷积模型的思想发展自图信号处理领域，理解图信号处理的相关内容是建立图卷积神经网络的最基础知识。故本部分的结构主要分为对图信号处理领域基础概念的定义，如图信号等等，接着类比信号处理中的傅里叶变换，得到图傅里叶变换的定义，并由此引出图信号频率的定义。最后介绍图信号上的滤波操作，将普通的图卷积、图滤波进行推广，得到图卷积模型。

（1）图信号

综合对比文献[19]、文献 [20]、文献[21]，我尝试给出如下关于图信号的定义：图信号是定义在图的节点上的信息，将它们数值化为向量（也可能退化为标量）后按照取定的节点顺序排列而成的矩阵（通常一行为一个节点的特征，当节点的特征退化为标量时，该矩阵退化为一个列向量）。

由此定义，可以发现图信号是定义在节点上的信号，而节点之间有自己固有的关联结构。在研究图信号性质的时候，除了要考虑图信号的强度之外，还需要考虑图的拓扑结构，不同图上同一强度的信号具有截然不同的性质。为此，有必要再引入一个描述图的结构性质的核心对象，它不仅需要能够准确全面且唯一确定地描述一个给定的图结构，还需要能方便地参与图信号的计算。因此，下面引入了拉普拉斯矩阵的概念。

（2）拉普拉斯矩阵与总变差

由于图卷积和卷积一样，要考虑到一个节点相邻节点的信息，并将它聚合体现在该节点下一层的更新特征上。这在卷积神经网络中是通过卷积核实现的。从卷积核及其计算形式上来看，卷积神经网络中的卷积核实际上可以看作一个参数可以学习的滤波器。由此我们可以联想到用于空间锐化滤波的拉普拉斯算子，它是 n 维欧式空间中的一个二阶微分算子：

$$\Delta f=\sum_{i=1}^{n}\frac{\partial^2 f}{\partial x_i^2}$$

在离散的二维空间中它就退化为图像处理中常用的边缘检测算子，作用原理如下：

$$\begin{aligned}\Delta f(x,y)&=\frac{\partial^2 f(x,y)}{\partial x^2}+\frac{\partial^2 f(x,y)}{\partial y^2}\\&=[(f(x+1,y)-f(x,y))-(f(x,y)-f(x-1,y))]+\\&\quad[(f(x,y+1)-f(x,y))-(f(x,y)-f(x,y-1))]\\&=[f(x+1,y)+f(x-1,y)+f(x,y+1)+f(x,y-1)]-4f(x,y)\end{aligned}$$

而在实际处理图像时，上式被表示成矩阵形式：$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$。

显而易见，此算子作用到图像上就是将邻居的 4 个像素的信息聚合到该像素上，描述了中心像素与其上、下、左、右四个像素的差异。

将这个思想推广到图卷积神经网络中，可以自然地想到使用一个算子来计算节点与其一阶邻居节点的差异，并将其聚合到该节点上作为更新值。那么如何找到这样的算子？这里我从计算形式与目标出发来寻找一个合适的算子。

之前已将图信号进行了相关定义，这里不妨先假设每个节点的特征为一个标量这一最简单的形式（当每个节点的特征为向量时计算过程是类似的），则一个图信号可以表示为一个列向量 $\boldsymbol{x}=[x_1,x_2,\cdots,x_N]^T$，其中 N 是图的节点总数，x_i 是第 i 个节点上的特征。

这里研究者希望找到一个算子可以将变换后，各个节点上的特征聚合了一阶邻居节点的特征，并提取出邻居节点与中心节点之间的差异，便于从一定程度上提取特征。由于神经网络对于矩阵友好，且矩阵是算子的常用形式，我们的目标就转化为找到一个矩阵 L，可以满足上述要求，亦即 $\boldsymbol{y}=L\boldsymbol{x}=[y_1,y_2,\cdots,y_N]^T$，其中 y_i 聚合了节点 v_i 的一阶邻居节点的特征信息，且最好以刻画差异的形式存在。也就是对于图中的任意一个节点 v_i 而言，变换后的新特征 y_i 要刻画该节点与一阶邻居节点的差异性，类比离散情况中的拉普拉斯算子来思考，我认为可以得到类比的变化结果：

$$y_i = x_i \cdot \mathrm{Deg}(x_i) - \sum_{v_j \in N(v_i)} x_j$$

整理成矩阵形式，可以得到：

$$
\boldsymbol{y}=\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}
$$

$$
=\begin{bmatrix} x_1\cdot \mathrm{Deg}(x_1)-\sum\limits_{v_j\in N(v_1)} x_j \\ x_2\cdot \mathrm{Deg}(x_2)-\sum\limits_{v_j\in N(v_2)} x_j \\ \vdots \\ x_N\cdot \mathrm{Deg}(x_N)-\sum\limits_{v_j\in N(v_N)} x_j \end{bmatrix}
$$

$$
=\begin{bmatrix} x_1\cdot \mathrm{Deg}(x_1) \\ x_2\cdot \mathrm{Deg}(x_2) \\ \vdots \\ x_N\cdot \mathrm{Deg}(x_N) \end{bmatrix}-\begin{bmatrix} \sum\limits_{v_j\in N(v_1)} x_j \\ \sum\limits_{v_j\in N(v_2)} x_j \\ \vdots \\ \sum\limits_{v_j\in N(v_N)} x_j \end{bmatrix}
$$

$$
=\begin{bmatrix} \mathrm{Deg}(x_1) & & & \\ & \mathrm{Deg}(x_2) & & \\ & & \cdots & \\ & & & \mathrm{Deg}(x_N) \end{bmatrix}\cdot \boldsymbol{x}-\begin{bmatrix} \sum\limits_{v_j\in N(v_1)} x_j \\ \sum\limits_{v_j\in N(v_2)} x_j \\ \vdots \\ \sum\limits_{v_j\in N(v_N)} x_j \end{bmatrix}
$$

为了将后一项也写成矩阵点乘图信号列向量的形式，研究者参考了图论中的邻接矩阵，定义如下：对于一个图，其邻接矩阵 A 为：

$$
A_{ij}=\begin{cases} 1, \text{ if } (v_i,v_j)\in E \\ 0, \text{ else} \end{cases}。
$$

这样就可以很容易地将上面减号后的那一项写成矩阵点乘图信号列向量的形式：

$$
\begin{bmatrix} \sum\limits_{v_j\in N(v_1)} x_j \\ \sum\limits_{v_j\in N(v_2)} x_j \\ \vdots \\ \sum\limits_{v_j\in N(v_N)} x_j \end{bmatrix}=A\boldsymbol{x}。
$$

再定义度矩阵 D 为：

$$D=\begin{bmatrix}\mathrm{Deg}(x_1) & & & \\ & \mathrm{Deg}(x_2) & & \\ & & \cdots & \\ & & & \mathrm{Deg}(x_N)\end{bmatrix},$$

则上式可以转化为：

$$\boldsymbol{y}=\begin{bmatrix}\mathrm{Deg}(x_1) & & & \\ & \mathrm{Deg}(x_2) & & \\ & & \ddots & \\ & & & \mathrm{Deg}(x_N)\end{bmatrix}\cdot\boldsymbol{x}-\begin{bmatrix}\sum\limits_{v_j\in N(v_1)} x_j \\ \sum\limits_{v_j\in N(v_2)} x_j \\ \vdots \\ \sum\limits_{v_j\in N(v_N)} x_j\end{bmatrix}=D\boldsymbol{x}-A\boldsymbol{x}\text{。}$$

回到最初的目标（找到一个矩阵 L，满足 $\boldsymbol{y}=L\boldsymbol{x}=[y_1,y_2,\cdots,y_N]^T$，其中 y_i 聚合了节点 v_i 的一阶邻居节点的特征信息，且最好以刻画差异的形式存在），现在就可以很清楚地看到矩阵 L 的结构形式，为 $L=D-A$。正如论文[19]一样，研究者将其定义为拉普拉斯矩阵。

拉普拉斯矩阵是用来研究图的结构性质的核心对象，拉普拉斯矩阵定义如下：$L=D-A$，D 是一个对角矩阵，D_{ii} 表示的是结点 v_i 的度；A 是邻接矩阵。

有了拉普拉斯矩阵，实际上就是将图的结构矩阵化了，这样图的结构信息就可以方便地和单个节点的孤立特征以一定的方式共同参与到计算中：

$$L\boldsymbol{x}=(D-A)\boldsymbol{x}=\left[\cdots,\sum_{v_j\in N(v_i)}(x_i-x_j),\cdots\right]\text{。}$$

由上式可以发现（也正如研究者的推导过程一样），在图信号中，拉普拉斯矩阵可以被用来描述中心节点与邻居节点之间的信号差异。由此可知，拉普拉斯矩阵是一个反应图信号局部平滑度的算子。进一步来说，拉普拉斯矩阵可以用来定义一个重要的二次型：

$$\boldsymbol{x}^T L\boldsymbol{x}=\sum_{v_i}\sum_{v_j\in N(v_i)} x_i(x_i-x_j)=\sum_{e_{ij}\in E}(x_i-x_j)^2$$

不妨令 $\mathrm{TV}(\boldsymbol{x})=\boldsymbol{x}^T L\boldsymbol{x}=\sum\limits_{e_{ij}\in E}(x_i-x_j)^2$，显然 $\mathrm{TV}(\boldsymbol{x})$ 对各条边上的信号的差值平方进行加和，可以用来刻画图信号整体的平滑度。本文沿用图信号处理中的一贯叫法 Total Variation，不妨译作“总变差”。

（3）图傅里叶变换与逆图傅里叶变换

傅里叶变换在数字信号处理中最为基础，它将信号从时域空间转换到频域空间，为信号的处理带来了极大的便利。同样，类比傅里叶变换，图傅里叶变换也是图信号处理的基石，它将图信号从空域视角转换到频域视角，便于图信号处理理论体系的建立。

假设图 G 的拉普拉斯矩阵为 L，由于 L 是一个实对称矩阵，根据实对称矩阵都可以被正交对角化，将其正交对角化可得：

$$L = V\Lambda V^T = \begin{bmatrix} \boldsymbol{v}_1 & \boldsymbol{v}_2 & \cdots & \boldsymbol{v}_N \end{bmatrix} \begin{bmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_N \end{bmatrix} \begin{bmatrix} \boldsymbol{v}_1^T \\ \boldsymbol{v}_2^T \\ \vdots \\ \boldsymbol{v}_N^T \end{bmatrix} 。$$

$V \in R^{N\times N}$ 是一个正交矩阵，即 $VV^T = I$。$V = [\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N]$ 表示 L 的 N 个特征向量，由于 V 是一个正交矩阵，这些特征向量都是彼此之间线性无关的单位向量。λ_k 表示第 k 个特征向量对应的特征值，为了后续处理方便，这里我们对特征值进行升序排列，即 $\lambda_1 \leqslant \lambda_2 \leqslant \ldots \leqslant \lambda_N$。

有了以上铺垫，下面我们就可以定义图傅里叶变换。对于任意一个在图 G 上的信号 $\boldsymbol{x}$，其图傅里叶变换为：

$$\boldsymbol{x}_{GFT_k} = \langle \boldsymbol{v}_k, \boldsymbol{x} \rangle 。$$

我们称特征向量为傅里叶基，$\boldsymbol{x}_{GFTk}$ 是 $\boldsymbol{x}$ 在第 k 个傅里叶基上的傅里叶系数。从此定义式上可以看到，傅里叶系数本质上是图信号在傅里叶基上的投影，由线性代数内容可知，它衡量了图信号与傅里叶基之间的相似度。将上式改写为矩阵形式，可以一次性计算出所有的傅里叶系数：

$$\boldsymbol{x}_{GFT} = V^T \boldsymbol{x} 。$$

由于 V 是一个正交矩阵，对上式左乘 V 可以得到：

$$V\boldsymbol{x}_{GFT} = VV^T \boldsymbol{x} = I\boldsymbol{x} = \boldsymbol{x} ,$$

亦即：

$$\boldsymbol{x} = V\boldsymbol{x}_{GFT} 。$$

类比逆傅里叶变换，将其定义为逆图傅里叶变换。

下面对其展开进行观察：

$$
\begin{aligned}
\boldsymbol{x} &= V\boldsymbol{x}_{GFT} \\
&= \begin{bmatrix} \boldsymbol{v}_1 & \boldsymbol{v}_2 & \cdots & \boldsymbol{v}_N \end{bmatrix} \begin{bmatrix} x_{GFT_1} \\ x_{GFT_2} \\ \vdots \\ x_{GFT_N} \end{bmatrix} \\
&= x_{GFT_1}\boldsymbol{v}_1 + x_{GFT_2}\boldsymbol{v}_2 + \cdots + x_{GFT_N}\boldsymbol{v}_N
\end{aligned}
$$

从线性代数的角度来看，$\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$组成了 N 维特征空间中的一组完备基向量，图 G 上任意一个图信号都可以被它们线性表出，权重就是图信号在对应傅里叶基上的傅里叶系数。

定义了图傅里叶变换后，下面就可以从频域视角研究图信号了。将图信号所有傅里叶系数合在一起称为该信号的频谱。

实际上，图信号频谱上的任意一个傅里叶系数都是对图信号某种低频或高频特征的定量描述，这种描述既考虑了节点各自的特征，也考虑了图的结构信息。

（4）图滤波器

为了引入图卷积神经网络，我们还需要定义图信号处理的一个核心概念——图滤波器。类比于离散信号处理，在图信号处理中，将图滤波器定义为对给定图信号的频谱中各个频率分量的强度进行增强或衰减的操作。假设图滤波器为 $H \in R^{N\times N}$，$H: R^N \rightarrow R^N$，令输出的图信号为 $\boldsymbol{y}$，则有：

$$
\boldsymbol{y} = H\boldsymbol{x} = \sum_{k=1}^{N} (h(\lambda_k) x_{GFT_k}) \boldsymbol{v}_k
$$

为看清 H 的形式，不妨对上式进行变换：

$$\boldsymbol{y}=H\boldsymbol{x}=\sum_{k=1}^{N}(h(\lambda_k)x_{GFT_k})\boldsymbol{v}_k=\begin{bmatrix}\boldsymbol{v}_1 & \boldsymbol{v}_2 & \cdots & \boldsymbol{v}_N\end{bmatrix}\begin{bmatrix}h(\lambda_1)x_{GFT_1}\\ h(\lambda_2)x_{GFT_2}\\ \vdots\\ h(\lambda_N)x_{GFT_N}\end{bmatrix}$$

$$=V\begin{bmatrix}h(\lambda_1) & & & \\ & h(\lambda_2) & & \\ & & \ddots & \\ & & & h(\lambda_N)\end{bmatrix}\begin{bmatrix}x_{GFT_1}\\ x_{GFT_2}\\ \vdots\\ x_{GFT_N}\end{bmatrix}$$

$$=V\begin{bmatrix}h(\lambda_1) & & & \\ & h(\lambda_2) & & \\ & & \ddots & \\ & & & h(\lambda_N)\end{bmatrix}V^T\boldsymbol{x}$$

将$\begin{bmatrix}h(\lambda_1) & & & \\ & h(\lambda_2) & & \\ & & \ddots & \\ & & & h(\lambda_N)\end{bmatrix}$记为$\Lambda_h$，则得到：

$$H=V\begin{bmatrix}h(\lambda_1) & & & \\ & h(\lambda_2) & & \\ & & \ddots & \\ & & & h(\lambda_N)\end{bmatrix}V^T=V\Lambda_h V^T$$

对比可以发现，H与拉普拉斯矩阵极为相似，只是对角线上的值不同，经过了一层函数的处理。可以清楚地看到对各个频率分量的强度进行增强或衰减的操作是通过$h(\lambda)$项来控制的。

（5）图卷积神经网络

至此，本文中所涉及的图信号处理基础已经全部定义或推导完毕。下面基于以上基础，建立图卷积神经网络模型。首先，先定义图信号处理中的卷积操作：

$$\boldsymbol{x}_1 * \boldsymbol{x}_2=\mathrm{IGFT}(\mathrm{GFT}(\boldsymbol{x}_1)\odot\mathrm{GFT}(\boldsymbol{x}_2))$$

其中$\odot$代表哈达玛积。

对上式继续进行推导，有：

$$
\begin{aligned}
\boldsymbol{x}_1 * \boldsymbol{x}_2 &= V((V^T \boldsymbol{x}_1) \odot (V^T \boldsymbol{x}_2)) \\
&= V(\boldsymbol{x}_{GFT_1} \odot (V^T \boldsymbol{x}_2)) \\
&= V(\mathrm{diag}(\boldsymbol{x}_{GFT_1})(V^T \boldsymbol{x}_2)) \\
&= (V\,\mathrm{diag}(\boldsymbol{x}_{GFT_1})V^T)\boldsymbol{x}_2
\end{aligned}
$$

显然$V\,\mathrm{diag}(\boldsymbol{x}_{GFT_1})V^T$可以视为一个图滤波器，其频率响应矩阵为$\boldsymbol{x}_1$的频谱。由此可知，两组图信号的图卷积运算总能转化为对应形式的图滤波运算，也就是说图卷积等价于图滤波。

根据论文[20]，图卷积神经网络中的图卷积运算都是指上式右边的通用的滤波形式，对应的卷积信号并不需要显式地给出，在图神经网络计算过程中，并不需要关心。

基于此，许多研究者给出了不同的研究结果，文献[19]、[20]、[21]是较为代表性的三种。下面详述之，并分析何种方式适合 Cora 数据集。

（1）参数化频率响应矩阵[19]

图卷积操作等价于图滤波操作，而图滤波算子的核心在于频率响应矩阵。我们可以联想到卷积神经网络中的卷积核实际上就是参数化了图像处理中的固定的滤波器，这些参数可以通过网络训练学习得到。那么我们自然而然可以想到对频率响应矩阵进行参数化，让频率响应矩阵对图数据的变换操作可以通过网络的训练学习得到，以使得对于图数据的变换可以满足我们的任务需求。根据文献[19],可以定义如下神经网络层：

$$
X' = \sigma\left(V \begin{bmatrix} \theta_1 & & & \\ & \theta_2 & & \\ & & \ddots & \\ & & & \theta_N \end{bmatrix} V^T X \right) = \sigma(\Theta X)
$$

其中，σ(·)是激活函数，$\theta = [\theta_1, \theta_2, \ldots, \theta_N]$是需要学习的参数，$\Theta$是对应的需要学习的图滤波器，$X$是输入的图信号矩阵，$X'$是输出的图信号矩阵。

此思路虽然简单易懂，但是也存在一个较大的问题：引入的学习参数过多，需要学习的参数量与图中的节点数一致，这在大规模图数据中极易发生过拟合问题。

另外，在真实的图数据中，数据的有效信息通常都蕴含在低频段中[20]，因此为图滤波器设置N个维度的自由度，且对每个频率都进行学习是没有必要的。

（2）频率响应矩阵的多项式拟合[21]

理论上，我们想要实现任意性质的图滤波器时，也就是实现任意类型函数曲线的频率响应函数，这时可以从多项式拟合中受到启发，通过逼近理论，我们可以用泰勒展开——多项式逼近函数去近似任意函数。结合上小节对图滤波器形式的分析，我们知道图滤波器 H 与拉普拉斯矩阵 L 极为相似，只是对角矩阵的对角线上的值不同，经过了一层函数 $h(\bullet)$ 的处理。这样我们很容易写出使用拉普拉斯矩阵 L 对图滤波器 H 进行多项式拓展的形式：

$$H = h_0 L^0 + h_1 L^1 + h_2 L^2 + \cdots + h_K L^K = \sum_{k=0}^{K} h_k L^k \text{ 。}$$

这实际上是通过多项式形式拓展来逼近频率响应函数 $h(\bullet)$ 。理论上，只要 K 够大，亦即项数越多，拟合的效果就越精确。

为了拟合任意的频率响应函数，除了（1）中直接将频率响应函数的对角线值进行参数化，我们也可以将拉普拉斯矩阵的多项式形式转化为一种可学习的形式，该思路在文献[21]中被提出，如下所示：

$$X' = \sigma\left(V\left(\sum_{k=0}^{K}\theta_k \Lambda^k\right)V^T X\right) = \sigma\left(V\,\mathrm{diag}(\psi\theta)V^T X\right)$$

其中 θ 是多项式系数向量，也是该网络层真正需要学习的参数，与前一个方法不同的是，这个方法的参数量 K 可以自由控制，一般设置 K<<N，这将大大降低模型过拟合的风险。

（3）图滤波器固定[20]

前一个方法虽然大大降低了参数量，但却由于对矩阵特征分解比较依赖而给计算带来了极高的复杂度。为了解决这个问题，文献[20]中对上式进行限制，设 K = 1，且令 $\theta_0 = \theta_1 = \theta$，则有：

$$X' = \sigma\left(\theta(I + L)X\right) = \sigma\left(\theta L' X\right)$$

此处 θ 是一个标量，相当于对 L'的频率响应函数做了一个尺度变换，通常这种尺度变换在神经网络模型中会被归一化操作替代，因此这个参数是不必要引入的，可以设 $\theta = 1$。

值得注意的是，经过这样转化后，虽然模型复杂性大大降低，但是我们发现我们将原有的所有可学习参数都固定了，也就是说网络更新时并没有可以更新的参数。因此为了加强网络的拟合能力，增强网络的可学习能力（引入可学

习参数），可以增加一个参数化的权重矩阵 W 对输入的图信号做一个仿射变换，于是最终得到图卷积层模型：

$$X' = \sigma\left(L'XW\right)$$

本 Cora 数据集实验中，我决定采取该模型作为图卷积层，并以此为主体堆叠多层，构成图卷积神经网络。

虽然该 GCN 层是由对频率响应函数的线性近似推导得出来的，但是和深度学习中深度网络给模型带来的强大拟合能力一样，堆叠多层 GCN 层，在某种程度上，可以达到高阶多项式形式的频率响应函数的滤波能力。这种简化单层网络的学习能力，通过增加深度来提升模型表达能力的做法，在 CNN 模型中表现出了极强的工程优越性。事实证明，这种设计所带来的优越性也在 GCN 后续的相关论文中得到了充分展示。

STEP 2 图卷积层的定义

```python
class GraphConvolution(nn.Module):
    def __init__(self, inDim, outDim):
        super(GraphConvolution, self).__init__()
        self.inDim = inDim
        self.outDim = outDim
        self.weight = nn.Parameter(torch.Tensor(inDim, outDim))
        init.kaiming_uniform_(self.weight)
        self.bias = nn.Parameter(torch.Tensor(outDim))
        init.zeros_(self.bias)

    def forward(self, A, inp):
        return torch.sparse.mm(A, torch.mm(inp, self.weight)) + self.bias
```

图 7 本实验定义的图卷积层

具体在本次基于 Cora 数据集的论文节点分类实验中，我基于上面得到的图卷积层模型，使用 PyTorch 构建了图卷积层，如图 7 所示，并搭建了一个三层的图卷积神经网络。具体的网络结构在第四部分“数据设计及预处理”小节结合数据的处理情况进行具体阐述。

四、编程实战：构建图卷积神经网络解决 Cora 数据集论文分类问题

完成了图卷积层的建模，我们就可以很容易地通过堆叠多层图卷积层完成图卷积神经网络的搭建，下面我们利用 PyTorch 搭建一个图卷积神经网络，完成本调研报告开始时想要完成的任务——论文主题分类，具体看一看图卷积神经网络是如何实际应用的。同时，为了分析图卷积神经网络对于图数据节点邻

居信息的聚合效果是否有效，我还建立了一个全连接神经网络，仅仅将孤立的论文文本特征输入，进行同样的主题分类任务，用于作为对照。

为了便于实验，我们采用 Cora 数据集进行实验。

（一）数据设计及预处理

由于本调研报告研究目标是论文主题分类，除了考虑每篇论文各自的孤立特征以外，还考虑了论文的引用关系，为了减少搜集与整理数据的时间，研究者直接使用公开数据集 Cora 数据集进行模型的测试。

Cora 数据集由 2708 篇人工智能相关论文组成，这些论文被分类进 7 个子主题中，包括神经网络、强化学习、规则学习、概率方法、遗传算法、理论研究、案例相关。各个论文之间一共存在 5429 个引用关系，由此构成了引用网。每篇论文的特征是通过词袋模型得到的，数据集制作者选取了 1433 个特殊词组成字典，每篇论文被编码为 1433 维的向量，每一维度对应一个特殊词。若一篇论文中出现了该特殊词，那么代表该词的维度就被置为 1，否则为 0。

数据预处理阶段，结合上面已经定义的图卷积神经网络的模型及运算，对原始数据进行相关的预处理操作。

由于在 Cora 数据集中，引用关系是通过邻接表的形式给出的，而图神经网络的计算需要使用邻接矩阵，因此我们首先要将邻接表转化为邻接矩阵。值得注意的是，由于论文数量繁多，论文的引用关系数相比论文数量来说还是比较小的，也就是说在这个引文关系图中，图中边的数量要比节点的数量小很多，这就使得最后邻接矩阵会成为一个稀疏矩阵。由于邻接矩阵是稀疏矩阵，使用普通矩阵来存储和计算明显是对存储空间和计算性能的浪费，故这里实验者将邻接矩阵存储为稀疏矩阵的形式（类型为 scipy.sparse.coo_matrix），使用稀疏矩阵的乘法来提高运算效率。最后通过对数据的处理，实验者得到了一个大小为 2708×2708 的邻接矩阵（2708 为总的样本数量），具体代码可参见附录代码中的“_adjacencyHelp(adjTable)”函数。

值得一提的是，为了增强网络学习过程中的数值稳定性，文献[19]的作者仿照了拉普拉斯矩阵的正则化过程，对刚刚定义的图卷积层 $X'=\sigma\left(L'XW\right)$ 中的 $L'=I+L$ 也进行了归一化处理。

即令 $L_{\text{sym}}'=D'^{-0.5}A'D'^{-0.5}$,其中 $A'=A+I$ ， $D_{ii}'=\sum_j A_{ij}'$ 。

则定义的图卷积层变成 $X'=\sigma\left(L_{\text{sym}}'XW\right)$ 。

因此转化得到该引文网络图的邻接矩阵后，还需要对其按如上式子进行归一化，具体过程见附录代码的“normalization(A)”函数。

正如很多神经网络的学习任务一样，我们的最终目的是将训练好的模型部署到真实的环境中，希望训练好的模型在真实的数据上得到较好的预测结果。一般将模型在真实环境中的误差称为泛化误差，网络学习的最终目的是让训练好的模型泛化误差越低越好，如果仅仅在训练集上误差很低而泛化误差却较高的话，这个模型并没有很大的价值。要解决这个问题，通常的做法是将数据集分割为训练集、测试集和验证集。使用训练集的数据来训练这个网络模型，然后使用测试集上的误差作为最终模型在应对现实中的泛化误差的模拟。要使泛化误差小，只需要使测试集上的误差小即可。同时为了便于调节参数，再划分出一个验证集，认为在验证集上使得误差更小的模型更优，避免使用测试集选择模型参数时导致测试集信息泄露。

接下来读入节点特征与标签，由于一个节点代表一篇论文，每篇论文的特征是通过词袋模型得到的一个 1433 维的向量。再加上有 2708 篇文章，最后得到的节点特征矩阵大小为 2708×1433，标签大小为 2708×1，其中每个论文的标签是一个 0 ~ 6 的整数，指示本篇论文属于哪个类别（子主题）。

同时，我为了进一步确保网络学习过程的稳定，将每个节点的 1433 维度的特征向量也进行了归一化，使得节点特征矩阵的每一行的和为 1。

本实验中，我进行了对比实验，除了本文研究的图卷积神经网络外，还设计了传统的全连接神经网络进行性能对比分析。为了直观看出图卷积神经网络与普通全连接神经网络（不考虑引用关系，仅仅考虑各个论文孤立的文本特征信息）的性能差异，实验者选取了较小的训练集样本数，即：训练集样本数 140、验证集样本数 500、测试集样本数 1000。对于神经网络而言，训练集如此小的情况下，训练学习过程是有挑战性的，也更能看出网络的性能如何。

由于上一部分设计模型时还未清楚数据维度等信息，尚且不能确定网络的输入输出大小。至此得到了数据的情况，因此就可以根据数据来设计网络的输入与输出大小。根据图 7 定义的图卷积层，这里希望根据 1433 维度的节点特征得到该节点代表论文的主题分类。由于一共有 7 个主题分类，联想类比 MNIST、SVHN 等手写体数字识别问题，不妨将最后的输出维度定为 7，选取该 7 维向量元素最大的那个维度对应的类别作为该节点论文的主题分类。

于是设计网络如下图 8 所示：

STEP 3 图卷积神经网络的定义

```
class GCN(nn.Module):
    def __init__(self):
        super(GCN, self).__init__()
        self.gcn1 = GraphConvolution(1433, 500)
        self.gcn2 = GraphConvolution(500, 100)
        self.gcn3 = GraphConvolution(100, 7)

    def forward(self, A, inp):
        outp1 = F.relu(self.gcn1(A, inp))
        outp2 = F.relu(self.gcn2(A, outp1))
        outp3 = self.gcn3(A, outp2)
        return outp3
```

图 8 本实验定义的图卷积神经网络

如图所示，一共定义了 3 层图卷积层，其中第一层输入维度 1433，输出维度 500；第二层隐藏层输入维度 500，输出维度 100；第三层输出层输入维度 100，输出维度 7。

至此数据处理与网络设计已经完成，后面只需要使用反向传播算法进行网络的学习即可。

（二）实验及实验结果分析

具体实验中，实验者设计了对比实验，除了本文研究的图卷积神经网络（网络结构与参数上面的部分已经详述）以外，还设计了一个传统的 4 层全连接神经网络，具体结构为四层神经元数量分别为：1433、500、100、7，激活函数使用 ReLU 激活函数。

与图卷积神经网络相比，该全连接神经网络的输入为单篇论文的 1433 维度的特征向量，监督标签为该论文所属的主题/类别。也就是说，该全连接神经网络不考虑论文的引用关系，依旧采用较为传统的分类思想。

而此图卷积神经网络，通过引入拉普拉斯矩阵等描述引文网络的拓扑图结构的矩阵参与计算，实际上在考虑了单篇论文的孤立特征之外，还考虑了论文之间的引用关系，拥有更多的对该节点（论文）的认知。

在实际操作中，两个网络的学习率都设为 0.1，权重衰减设为 5e-4，都学习 200 个 epoch。数据集划分均为训练集样本数 140、验证集样本数 500、测试集样本数 1000。均使用 Adam 优化算法。

下面分析具体的实验结果。

如下图 9、图 10、图 11、图 12 分别是此图卷积神经网络在 140 个样本大小的训练集上进行学习的日志输出（代价函数输出、训练集上准确率、验证集上准确率）、测试集上准确率随 epoch 数的变化图、测试集上代价函数输出随 epoch 数的变化图、对特征分类结果进行 TSEN 降维可视化图。

图 13、图 14、图 15 分别是用于对照的全连接神经网络在 140 个样本大小的训练集上进行学习的日志输出（代价函数输出、训练集上准确率、验证集上准确率）、测试集上准确率随 epoch 数的变化图、测试集上代价函数输出随 epoch 数的变化图。

```
Epoch 187: Loss 0.0277, TrainAcc 1.0000, ValAcc 0.8080
Epoch 188: Loss 0.0267, TrainAcc 1.0000, ValAcc 0.8060
Epoch 189: Loss 0.0275, TrainAcc 1.0000, ValAcc 0.8080
Epoch 190: Loss 0.0266, TrainAcc 1.0000, ValAcc 0.8060
Epoch 191: Loss 0.0273, TrainAcc 1.0000, ValAcc 0.8100
Epoch 192: Loss 0.0266, TrainAcc 1.0000, ValAcc 0.8080
Epoch 193: Loss 0.0269, TrainAcc 1.0000, ValAcc 0.8040
Epoch 194: Loss 0.0273, TrainAcc 1.0000, ValAcc 0.8120
Epoch 195: Loss 0.0266, TrainAcc 1.0000, ValAcc 0.8100
Epoch 196: Loss 0.0266, TrainAcc 1.0000, ValAcc 0.8060
Epoch 197: Loss 0.0270, TrainAcc 1.0000, ValAcc 0.8120
Epoch 198: Loss 0.0266, TrainAcc 1.0000, ValAcc 0.8080
Epoch 199: Loss 0.0264, TrainAcc 1.0000, ValAcc 0.8080
Test acc:  0.8100000023841858
```

图 9

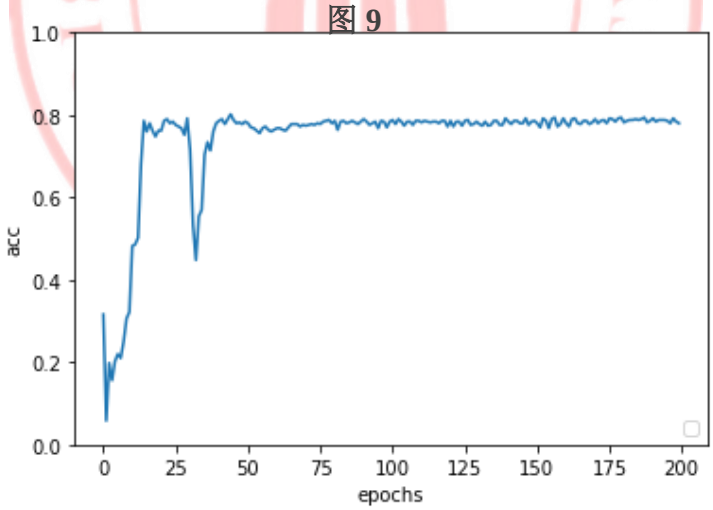

图 10

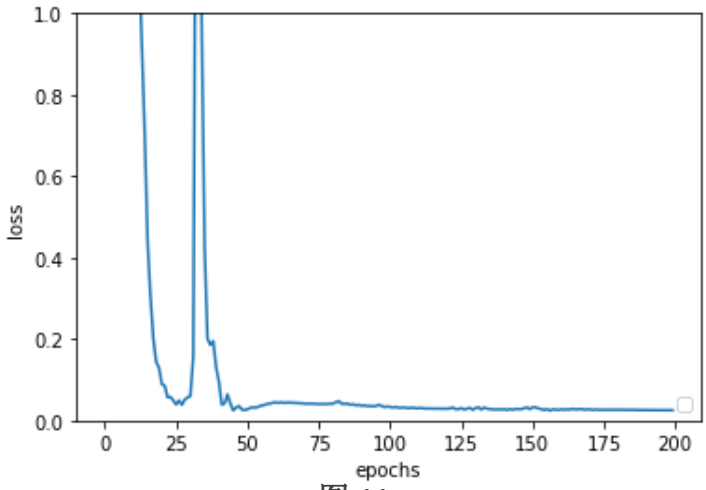

图 11

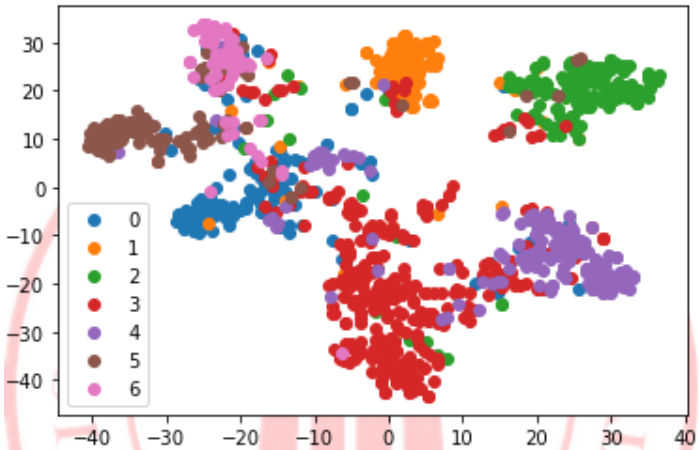

图 12

```
Epoch 190: Loss 0.0110, TrainAcc 1.0000, ValAcc 0.4920
Epoch 191: Loss 0.0114, TrainAcc 1.0000, ValAcc 0.5040
Epoch 192: Loss 0.0115, TrainAcc 1.0000, ValAcc 0.4940
Epoch 193: Loss 0.0115, TrainAcc 1.0000, ValAcc 0.4960
Epoch 194: Loss 0.0114, TrainAcc 1.0000, ValAcc 0.4940
Epoch 195: Loss 0.0115, TrainAcc 1.0000, ValAcc 0.4980
Epoch 196: Loss 0.0114, TrainAcc 1.0000, ValAcc 0.4960
Epoch 197: Loss 0.0114, TrainAcc 1.0000, ValAcc 0.4860
Epoch 198: Loss 0.0115, TrainAcc 1.0000, ValAcc 0.4920
Epoch 199: Loss 0.0114, TrainAcc 1.0000, ValAcc 0.4960
Test acc:  0.4959999918937683
```

图 13

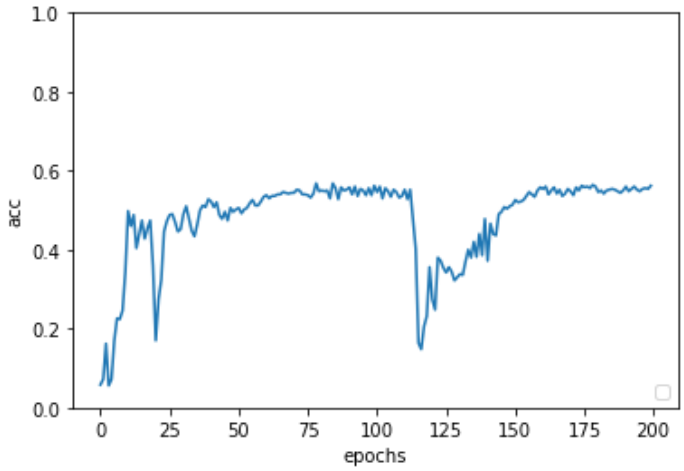

图 14

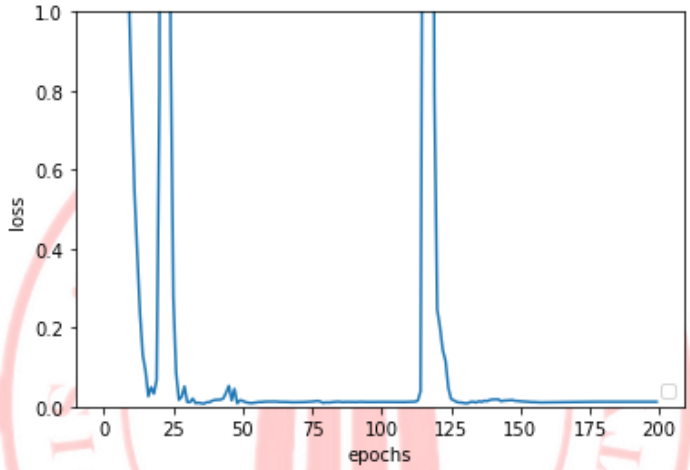

图 15

首先可以看到，不论是图卷积神经网络还是全连接神经网络，模型全部收敛，说明这两种网络至少都是有效果的。两个网络的训练过程中，测试集上准确率总体都呈上升趋势；代价函数输出总体都呈下降趋势。

观察训练过程的日志输出可以看出，图卷积神经网络和全连接神经网络在训练集上的准确率在训练的最后阶段均稳定到了 100%，但是值得注意的是，图卷积神经网络在验证集上的准确率在训练的最后阶段稳定在 80%左右，而全连接神经网络在验证集上的准确率在训练的最后阶段只能稳定在 50%左右。图卷积神经网络比传统全连接网络高出整整 30%！

除了验证集准确率，测试集上准确率也能看出图卷积神经网络的分类效果（测试集准确率约 81%）比传统全连接神经网络（测试集准确率约 49.6%）要好很多。

上面两点说明，我们在一开始的想法是正确的，即除了论文文本自身的特征外，论文这一特殊文本类型的特殊信息——引用关系，对于论文的主题分类也能提供有效的信息，大大提升模型分类准确率。

另外，可见相对测试集样本量，训练集样本量极小的情况下，图卷积神经网络模型依旧达到了近 80%的测试集上准确度。说明图卷积神经网络不仅继承了其他神经网络共有的表达能力强的优势，同时对于图数据的确有着良好的适配性，且泛化能力较好，有望在其他图数据，如社交网络分析、药物分子合成等等方面有进一步的应用。

五、总结与讨论

本调研报告从数学原理出发，针对卷积神经网络等传统神经网络的局限性，类比卷积神经网络，自然地引入了图卷积神经网络这一全新的神经网络，并将其应用在论文特征与引用关系数据集 Cora 数据集上，完成论文的主题分类任务，并搭建训练一个传统全连接神经网络完成同样的分类任务作为对照实验，以对比观察图卷积机制对于图数据的学习效果。

本调研报告的实验基于“将图神经网络运用于论文主题分类中，考虑孤立的论文自身特征，同时考虑论文的引用关系，会获得比先前两种考量单独实践的效果更好的效果”这一猜想，另辟蹊径，尝试突破此前国内外论文主题分类相关研究仅仅关注论文文本孤立特征或仅仅关注引用关系这一局限，将论文文本特征与论文引用关系结合考虑，设计论文（子）主题分类模型。

首先我将该问题转化为一个图信号处理问题，结合当下热门的图卷积神经网络，尝试将引文网络图数据引入神经网络模型中进行学习。为解决已有网络模型不适配图数据的问题，我从图信号处理开始探究，将图信号处理中的知识推广，并合理地类比卷积神经网络，结合相关文献，最后推导出了适用于图数据的图卷积神经网络模型，并实际搭建图卷积神经网络进行论文主题分类问题的训练。通过与传统全连接神经网络对比研究，我证实了本研究的最初猜想的正确性，将论文引用关系和论文孤立特征同时考虑，的确能获得比仅仅考虑论文孤立特征的模型更好的主题分类结果。基于此，我利用建立的图卷积神经网络模型，构建了一个令人满意的论文（子）主题自动分类方案。

除此任务本身以外，在实验过程中我还发现了当训练集上样本量极小的情况下，图卷积神经网络依然能学习到相比于其他神经网络模型更好的结果，说明图卷积神经网络不仅继承了其他神经网络共有的表达能力强的优势，同时对于图数据的确有着良好的适配性，且泛化能力较好，有望在其他图数据，如社交网络分析、药物分子合成等等方面有进一步的应用。

当然，本调研报告设计的方法还有许多值得改进的地方。例如本实验使用的 Cora 数据集，其论文的特征向量是通过词袋模型得到的。词袋模型是一种较为传统的文本特征提取方法，虽然实现简单，只需要统计文本中某些单词是否出现或出现次数，但是正因如此，它不考虑句子中单词的顺序，不考虑词与词之间的顺序和联系，所以有可能会导致句意表达不准确，进而导致用这种方式得到的文本特征向量不准确。后续的研究可以尝试改进论文的特征表示方法，例如采取 n 元特征（N-gram 特征）将句子的顺序考虑进去等等，使得论文的特征表示更加准确，有望获得更好的分类效果。

另外，虽然图卷积网络在近些年取得了成功，但仍然有些没有克服的问题和值得深入研究的方向。

例如，传统的深度学习模型，在堆叠了大量网络层后， 由于其强大的表示能力，在很多问题上了取得了显著的效果。但是在图卷积神经网络模型中，在堆叠了较少的层数后，网络就达到最好效果，同时再增加图卷积层反而会使得结果变差．因为图卷积包含了聚合邻居节点特征的操作，当网络堆叠多层后使得节点之间的特征过于平滑，缺乏区分性。GCN 的实验结果显示[10]，当网络层数超过两层后，随着层数增加，GCN 在半监督节点分类问题上的效果反而会下降。同时随着网络的不断叠加，最终所有的节点会学到相同的表达。图神经网络是否需要深层的结构，或者说能否设计出一种深层的网络结构能避免过于平滑的问题，是一个迫切需要解决的研究问题。

其次，在实际场景中，网络的规模往往非常大，比如新浪微博、Twitter 等社交关系网络，往往包含了数亿级的节点和边。而目前绝大部分的图卷积神经网络模型都不适用于这种大规模的网络．比如基于谱方法的图卷积神经网络需要计算图拉普拉斯矩阵的特征向量矩阵，而这个操作的计算复杂度和空间复杂度都很高，难以用于大规模网络。而空间方法，在更新节点表达时依赖于大量的邻居节点，也使得计算代价过大，不适用于大规模网络。虽然近些年已经有一些基于采样的方法来处理大规模网络数据的问题[22, 23, 24]，但是这一问题仍然没有得到有效解决。

当然，正如所有神经网络模型一样，图神经网络的可解释性也是难以攻克的一个难关。深度学习模型的可解释性与可视化一直是深度学习领域备受关注的方向，图结构给图神经网络的可解释性与可视化带来了新的挑战。如何可视化图神经网络学到的结构模式对于理解图神经网络的工作原理有重要意义[25]。图神经网络已经在很多场景下取得了显著的效果，Xu 等人[26]尝试给出了一 种解

释，但是如何从理论上说明图神经网络为什么能取得显著的提升仍然是一个没有解决的问题。

总的来说，目前图卷积神经网络取得了一定的效果，但是仍然有一些问题需要解决，相信在未来的研究中，图卷积神经网络会在图数据的场景中被进一步更加充分地使用，这些问题也能够得到合理且巧妙的解决。

参考文献

[1] Page L, Brin S, Motwani R, et al. The PageRank citation ranking: Bringing order to the web[R]. Stanford InfoLab, 1999.

[2] Kleinberg J M. Hubs, authorities, and communities[J]. ACM computing surveys (CSUR), 1999, 31(4es): 5-es.

[3] Ng A Y, Jordan M I, Weiss Y. On spectral clustering: Analysis and an algorithm[C]//Advances in neural information processing systems. 2002: 849-856.

[4] Qi J, Liang X, Li Z, et al. Representation learning of large-scale complex information network: Concepts, methods and challenges[J]. Chin. J. Comput, 2018, 41(10): 2394-2420.

[5] Perozzi B, Al-Rfou R, Skiena S. Deepwalk: Online learning of social representations[C]//Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining. 2014: 701-710.

[6] Tang J, Qu M, Wang M, et al. Line: Large-scale information network embedding[C]//Proceedings of the 24th international conference on world wide web. 2015: 1067-1077.

[7] Grover A, Leskovec J. node2vec: Scalable feature learning for networks[C]//Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining. 2016: 855-864.

[8] Bruna J, Zaremba W, Szlam A, et al. Spectral networks and locally connected networks on graphs[J]. arXiv preprint arXiv:1312.6203, 2013.

[9] Defferrard M, Bresson X, Vandergheynst P. Convolutional neural networks on graphs with fast localized spectral filtering[J]. Advances in neural information processing systems, 2016, 29: 3844-3852.

[10] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[J]. arXiv preprint arXiv:1609.02907, 2016.

[11] Synnestvedt M B, Chen C, Holmes J H. CiteSpace II: visualization and knowledge discovery in bibliographic databases[C]. AMIA Annual Symposium Proceedings. American Medical Informatics Association, 2005, 2005: 724.

[12] 杨清, 杨岳湘, 瞿国平. 智能文本分类系统的研究与设计[D]. 计算机应用研究, 1999, 10: 15-17.

[13] Shui-geng Z, Ji-hong G, Yan-xiang H. Hierarchical classification of Chinese documents based onN-grams[J]. Wuhan University Journal of Natural Sciences, 2001, 6(1-2): 416-422.

[14] Deerwester S C, Ziff D A, Waclena K. An architecture for full text retrieval systems[C]. Database and Expert Systems Applications. Springer, Vienna, 1990: 22-29.

[15] Hofmann T. Probabilistic latent semantic indexing[C]. Proceedings of the 22nd annual international ACM SIGIR conference on Research and development in information retrieval. 1999: 50-57.

[16] Blei D M, Ng A Y, Jordan M I. Latent dirichlet allocation[J]. the Journal of machine Learning research, 2003, 3: 993-1022.

[17] Zhang Xiao-dan. Application of taking literature classification of NSTL technology journals as an example [J]. Intelligence Journal, 2021, 40 (1): 184-188.

[18] Tai K S, Socher R, Manning C D. Improved semantic representations from tree-structured long short-term memory networks[J]. arXiv preprint arXiv:1503.00075, 2015.

[19] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[J]. arXiv preprint arXiv:1609.02907, 2016.

[20] Bruna J, Zaremba W, Szlam A, et al. Spectral networks and locally connected networks on graphs[J]. arXiv preprint arXiv:1312.6203, 2013.

[21] Defferrard M, Bresson X, Vandergheynst P. Convolutional neural networks on graphs with fast localized spectral filtering[J]. Advances in neural information processing systems, 2016, 29: 3844-3852.

[22] Hamilton W, Ying Z, Leskovec J. Inductive representation learning on large graphs[J]. Advances in neural information processing systems, 2017, 30.

[23] Ying R, He R, Chen K, et al. Graph convolutional neural networks for web-scale recommender systems[C]//Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2018: 974-983.

[24] Chen J, Ma T, Xiao C. Fastgcn: fast learning with graph convolutional networks via importance sampling[J]. arXiv preprint arXiv:1801.10247, 2018.

[25] Ying R, Bourgeois D, You J, et al. Gnn explainer: A tool for post-hoc explanation of graph neural networks[J]. arXiv preprint arXiv:1903.03894, 2019.

[26] Xu K, Hu W, Leskovec J, et al. How powerful are graph neural networks?[J]. arXiv preprint arXiv:1810.00826, 2018.

附录：实验代码

STEP 0 导入需要使用的库

```
import itertools
import os
import pickle
import urllib
from collections import namedtuple
import numpy as np
import scipy.sparse as sp
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.nn.init as init
import torch.optim as optim
import matplotlib.pyplot as plt
```

STEP 1 数据准备

```
dataTuple = namedtuple('dataTuple', ['x', 'y', 'adjacency', 'train_mask', 'val_mask', 'test_mask'])


class Cora(object):
    def __init__(self):
        self.filenames = ['ind.cora.x', 'ind.cora.tx', 'ind.cora.allx', 'ind.cora.y', 'ind.cora.ty', 'ind.cora.ally',
'ind.cora.graph', 'ind.cora.test.index']
        self.cachePath = "./cora/cachedCora.pkl"
        if os.path.exists(self.cachePath):
            self._cora = pickle.load(open(self.cachePath, "rb"))
        else:
            self._cora = self.cacheData()
        print("读入完毕！ ")


    def cacheData(self):
        tx = self._loadDataHelp("./cora/ind.cora.tx")
        allx = self._loadDataHelp("./cora/ind.cora.allx")
        y = self._loadDataHelp("./cora/ind.cora.y")
        ty = self._loadDataHelp("./cora/ind.cora.ty")
        ally = self._loadDataHelp("./cora/ind.cora.ally")
        graph = self._loadDataHelp("./cora/ind.cora.graph")
```

```
        testIndex = self._loadDataHelp("./cora/ind.cora.test.index")

        trainIndex = np.arange(y.shape[0])
        validIndex = np.arange(y.shape[0], y.shape[0] + 500)
        sortedTestIndex = sorted(testIndex)

        x = np.concatenate((allx, tx), axis=0)
        y = np.concatenate((ally, ty), axis=0).argmax(axis=1)

        x[testIndex] = x[sortedTestIndex]
        y[testIndex] = y[sortedTestIndex]
        numNodes = x.shape[0]

        trainMask = np.zeros(numNodes, dtype=bool)
        validMask = np.zeros(numNodes, dtype=bool)
        testMask = np.zeros(numNodes, dtype=bool)
        trainMask[trainIndex] = True
        validMask[validIndex] = True
        testMask[testIndex] = True
        adjacency = self._adjacencyHelp(graph)
        print("节点特征矩阵大小:", x.shape)
        print("标签大小:", y.shape)
        print("邻接矩阵大小:", adjacency.shape)
        print("训练集样本数:", trainMask.sum(),"验证集样本数:", validMask.sum(), "测试集样本数:",testMask.sum())
        res = dataTuple(x, y, adjacency, trainMask, validMask, testMask)
        with open(self.cachePath, "wb") as f:
            pickle.dump(res, f)
        print("数据已缓存！")
        return res

    def cora(self):
        return self._cora

    def _loadDataHelp(self,path):
        filename = os.path.basename(path)
        res = 0
        if(filename=="ind.cora.test.index"):
            res = np.genfromtxt(path,dtype="int64")
```

```
        else:
            res = pickle.load(open(path, "rb"), encoding="latin1")
            if(str(type(res))[8:13]=='scipy'):
                res = res.toarray()
        return res

    def _adjacencyHelp(self, adjTable):
        # build adjacency matrix from adjacency table
        edge = []
        nodeNum = len(adjTable)
        for sor, des in adjTable.items():
            edge.extend([sor, i] for i in des)
            edge.extend([i, sor] for i in des)
        edge = list(k for k, _ in itertools.groupby(sorted(edge)))
        edge = np.asarray(edge)
        A = sp.coo_matrix((np.ones(len(edge)), (edge[:,0],edge[:,1])),shape=(nodeNum,
nodeNum),dtype="float32")
        return A

    def normalization(self, A):
        A += sp.eye(A.shape[0])
        # add self link
        deg = np.array(A.sum(1))
        d = sp.diags(np.power(deg, -0.5).flatten())
        res = d.dot(A).dot(d).tocoo()
        return res

dataset = Cora().cora()
```

STEP 2 图卷积层的定义

```
class GraphConvolution(nn.Module):
    def __init__(self, inDim, outDim):
        super(GraphConvolution, self).__init__()
        self.inDim = inDim
        self.outDim = outDim
        self.weight = nn.Parameter(torch.Tensor(inDim, outDim))
        init.kaiming_uniform_(self.weight)
        self.bias = nn.Parameter(torch.Tensor(outDim))
        init.zeros_(self.bias)
```

```
    def forward(self, A, inp):
        return torch.sparse.mm(A, torch.mm(inp, self.weight)) + self.bias
```

STEP 3 图卷积神经网络的定义

```
class GCN(nn.Module):
    def __init__(self):
        super(GCN, self).__init__()
        self.gcn1 = GraphConvolution(1433, 500)
        self.gcn2 = GraphConvolution(500, 100)
        self.gcn3 = GraphConvolution(100, 7)

    def forward(self, A, inp):
        outp1 = F.relu(self.gcn1(A, inp))
        outp2 = F.relu(self.gcn2(A, outp1))
        outp3 = self.gcn3(A, outp2)
        return outp3
```

STEP 4 图卷积神经网络的训练

```
# some hyper parameters
learning_rate = 0.1
weight_decay = 5e-4
epochs = 200

# load data
dataset = Cora().cora()
x = torch.from_numpy(dataset.x/dataset.x.sum(1, keepdims=True))
y = torch.from_numpy(dataset.y)
train_mask = torch.from_numpy(dataset.train_mask)
val_mask = torch.from_numpy(dataset.val_mask)
test_mask = torch.from_numpy(dataset.test_mask)

adjacencyN = Cora().normalization(dataset.adjacency)
ind = torch.from_numpy(np.asarray([adjacencyN.row,adjacencyN.col]).astype('int64')).long()
val = torch.from_numpy(adjacencyN.data.astype(np.float32))
A = torch.sparse.FloatTensor(ind, val, (2708, 2708))

# realize model
model = GCN()
```

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate, weight_decay=weight_decay)

def train():
    lossList = []
    accList = []
    model.train()
    train_y = y[train_mask]
    for epoch in range(epochs):
        # forward computing
        forwOut = model(A, x)
        train_forwOut = forwOut[train_mask]
        loss = criterion(train_forwOut, train_y)
        optimizer.zero_grad()
        # backpropagation
        loss.backward()
        # weight update
        optimizer.step()
        train_acc, _, _ = test(train_mask)
        val_acc, _, _ = test(val_mask)
        # save acc and loss
        accList.append(val_acc.item())
        lossList.append(loss.item())
        print("Epoch {:03d}: Loss {:.4f}, TrainAcc {:.4f}, ValAcc {:.4f}".format(epoch, lossList[-1],
train_acc.item(), accList[-1]))
    return lossList, accList

def test(t_mask):
    model.eval()
    with torch.no_grad():
        output = model(A, x)
        test_mask_output = output[t_mask]
        predict_y = test_mask_output.max(1)[1]
        acc = torch.eq(predict_y, y[t_mask]).float().mean()
    return acc, test_mask_output.cpu().numpy(), y[t_mask].cpu().numpy()

loss, acc = train()
test_acc, test_out, test_label = test(test_mask)
print("Test acc: ", test_acc.item())
```

STEP 5 画图

```
# loss vs epoch
def plotLoss(loss):
    x = np.arange(200)
    plt.plot(x, loss, markevery=2)
    plt.xlabel("epochs")
    plt.ylabel("loss")
    plt.ylim(0, 1.0)
    plt.legend(loc='lower right')
    plt.show()
plotLoss(loss)

# acc vs epoch
def plotAcc(acc):
    x = np.arange(200)
    plt.plot(x, acc, markevery=2)
    plt.xlabel("epochs")
    plt.ylabel("acc")
    plt.ylim(0, 1.0)
    plt.legend(loc='lower right')
    plt.show()
plotAcc(acc)

# 绘制测试数据的 TSNE 降维图
from sklearn.manifold import TSNE
tsne = TSNE()
out = tsne.fit_transform(test_out)
fig = plt.figure()
for i in range(7):
    indices = test_label == i
    x, y = out[indices].T
    plt.scatter(x, y, label=str(i))
plt.legend()
```

STEP 6 对比实验：全连接网络

```
class FCN(nn.Module):
    def __init__(self):
        super(FCN, self).__init__()
```

```
        self.fcn1 = nn.Linear(1433, 500)
        self.fcn2 = nn.Linear(500, 100)
        self.fcn3 = nn.Linear(100, 7)

    def forward(self, inp):
        outp1 = F.relu(self.fcn1(inp))
        outp2 = F.relu(self.fcn2(outp1))
        outp3 = self.fcn3(outp2)
        return outp3

# realize model
model2 = FCN()
criterion2 = nn.CrossEntropyLoss()
optimizer2 = optim.Adam(model2.parameters(), lr=learning_rate, weight_decay=weight_decay)

def train2():
    lossList = []
    accList = []
    model2.train()
    train_y = y[train_mask]
    for epoch in range(epochs):
        # forward computing
        forwOut = model2(x)
        train_forwOut = forwOut[train_mask]
        loss = criterion2(train_forwOut, train_y)
        optimizer2.zero_grad()
        # backpropagation
        loss.backward()
        # weight update
        optimizer2.step()
        train_acc, _, _ = test2(train_mask)
        val_acc, _, _ = test2(val_mask)
        # save acc and loss
        accList.append(val_acc.item())
        lossList.append(loss.item())
        print("Epoch {:03d}: Loss {:.4f}, TrainAcc {:.4f}, ValAcc {:.4f}".format(epoch, lossList[-1],
train_acc.item(), accList[-1]))
    return lossList, accList

def test2(t_mask):
```

```
    model2.eval()
    with torch.no_grad():
        output = model2(x)
        test_mask_output = output[t_mask]
        predict_y = test_mask_output.max(1)[1]
        acc = torch.eq(predict_y, y[t_mask]).float().mean()
    return acc, test_mask_output.cpu().numpy(), y[t_mask].cpu().numpy()

loss, acc = train2()
test_acc, _, _ = test2(test_mask)
print("Test acc: ", test_acc.item())
```